

PPA Payment Components — Consumer Integration Guide

About these packages

Two reusable Angular 17 standalone components for the PPA Pain001 / ISO 20022 payment maker/checker workflow. Drop them into any Angular 17+ application. Compatible with Angular 17, 18 and 19.

Maker package	@citi-icg-169779/payment-maker
Checker package	@citi-icg-169779/payment-checker
Version	v1.0.0
Angular	17+ (compatible with 18 and 19)
Registry	Citi Artifactory — npm-icg-teamdev-local

What you get

Package	Selector	Description
@citi-icg-169779/payment-maker	<pm-maker-form>	Editable Pain001 form with 8 sections. Validates 12 required fields. Debounced hardcap check on Transaction Amount. Submit button enabled only when all fields valid and hardcap passes. Emits form data and validity to parent. Parent makes all API calls.
@citi-icg-169779/payment-checker	<pc-checker-form>	Read-only view of the maker form. Renders only when parent sets [visible]="true". Loads data via GET API. Only Transaction Amount is editable. Checker Approves or Rejects. Amount mismatch POST fires automatically if amount is modified before Approve.

Step 1 — Configure .npmrc

Add the following to your **project** .npmrc file:

```
.npmrc
```

```
# Scoped registry for @citi-icg-169779 packages
@citi-icg-169779:registry=https://www.artifactrepository.citigroup.net/artifactory/api/
npm/npm-icg-teamdev-local/
//www.artifactrepository.citigroup.net/artifactory/api/npm/npm-icg-teamdev-
local/:_authToken=${NPM_AUTH_TOKEN}
```

Step 2 — Authenticate

Set your Citi Artifactory token as an environment variable.

Windows PowerShell

```
$env:NPM_AUTH_TOKEN = "eyJ..." # paste your full Citi Artifactory token
```

macOS / Linux

```
export NPM_AUTH_TOKEN="eyJ..."
```

Most reliable approach on Windows Git Bash

Add the token directly to `C:\Users\<SOEID>\.npmrc` to avoid setting the environment variable every session.

Verify authentication

```
pnpm whoami --registry=https://www.artifactrepository.citigroup.net/artifactory/api/npm/
npm-icg-teamdev-local/
# Expected output: your Citi SOEID
```

Step 3 — Install

```
pnpm install @citi-icg-169779/payment-maker
pnpm install @citi-icg-169779/payment-checker
```

Step 4 — Project Setup

Add Bootstrap to angular.json

```
"styles": [  
  "node_modules/bootstrap/dist/css/bootstrap.min.css",  
  "src/styles.scss"  
]
```

Add provideHttpClient to main.ts

```
import { provideHttpClient } from '@angular/common/http';  
  
bootstrapApplication(AppComponent, {  
  providers: [provideHttpClient()]  
});
```

Step 5 — Use payment-maker

Architecture — parent drives the form

The maker component is a pure presenter. It emits data to the parent via outputs. The parent makes all API calls (hardcap check and submit). The submit button lives in the parent and is enabled/disabled based on the (formValidityChange) output.

Component class

```
import { Component, ViewChild } from '@angular/core';  
import { CommonModule } from '@angular/common';  
import {  
  MakerFormComponent,  
  PaymentComponentInput,  
  MakerSubmitResponse,  
  MakerApiService,  
  HardcapCheckResponse,  
  AmountChangeEvent,  
  Pain001Model,  
  FormFieldConfig,  
  DEFAULT_FIELD_CONFIG  
} from '@citi-icg-169779/payment-maker';  
  
@Component({  
  standalone: true,  
  imports: [CommonModule, MakerFormComponent]  
})  
export class YourComponent {
```

```

@ViewChild(MakerFormComponent) makerForm!: MakerFormComponent;

isFormValid = false;
isSubmitting = false;
hardcapResult: HardcapCheckResponse | null = null;

paymentInput: PaymentComponentInput = {
  applicationName: 'YOUR_APP',
  applicationModule: 'YOUR_MODULE',
  region: 'US',
  useMockApi: true,
  makerSubmitUrl: 'https://your-api.com/api/v1/pain001/maker/submit',
  hardcapCheckUrl: 'https://your-api.com/api/v1/pain001/hardcap/check',
  headers: { 'Authorization': 'Bearer your-token' }
};

fieldConfig: FormFieldConfig[] = DEFAULT_FIELD_CONFIG;

constructor(private apiService: MakerApiService) {}

onFormValidityChange(isValid: boolean): void {
  this.isFormValid = isValid;
}

onAmountChange(event: AmountChangeEvent): void {
  this.apiService.checkHardcap(
    {
      amount: event.amount,
      currencyCode: event.currencyCode,
      applicationName: this.paymentInput.applicationName,
      applicationModule: this.paymentInput.applicationModule
    },
    this.paymentInput
  ).subscribe({
    next: (res: HardcapCheckResponse) => {
      this.hardcapResult = res;
    },
    error: () => { this.hardcapResult = null; }
  });
}

onSubmitClick(): void {
  this.makerForm.onSubmit();
}

onFormSubmit(payload: Pain001Model): void {
  this.isSubmitting = true;
  this.apiService.submitMakerForm(payload, this.paymentInput).subscribe({
    next: (res: MakerSubmitResponse) => {
      this.isSubmitting = false;
      this.makerForm.onSubmitSuccess(res);
      this.showChecker = true;
    },
    error: (err: any) => {
      this.isSubmitting = false;
      this.makerForm.onSubmitError(err.message || 'Submission failed.');
```

```
    }  
  });  
}  
}
```

Quick Reference

PaymentComponentInput — payment-maker

Property	Type	Required	Description
applicationName	string	Yes	Sent as X-Application-Name header
applicationModule	string	Yes	Sent as X-Application-Module header
makerSubmitUrl	string	Yes	POST endpoint for Pain001 form submission

Need help?

- **Lightspeed CI/CD 403 error** — Build pipeline needs read access to npm-icg-teamdev-local.
- **Switching to real APIs** — Set useMockApi: false. No rebuild needed.
- **Hiding fields** — Use DEFAULT_FIELD_CONFIG.map() in your parent.